

Autonomous Exploration via Reinforcement Learning and Occupancy Grid Mapping

Colin Chong, Keng Xiang Tan, Humble Udohnyah

I. ABSTRACT - [COLIN CHONG, KENG XIANG TAN, HUMBLE UDOHNYAH]

Autonomous exploration is an important aspect for robots in terms of operating in disaster response or in planetary explorations. In these scenarios, prior knowledge of the environment is not available to the robot, and environments may be unpredictable. Hence, reinforcement learning and mapping will be crucial for the robot to succeed. This study will investigate the effectiveness of Tabular Q-Learning for unknown exploration, combining with Occupancy Grid Mapping (OGM) to record visited states and provide visualisation of coverage, then generalise the policy into similar but unknown environment. The agent in this study is trained on an isolated and fixed offline world with user-defined obstacles to confirm the fundamentals of Q-learning work. This setup ensures that it is reproducible and allowed us to clearly measure the impact of reward design and policy behaviour, before moving to a more complex online scenarios driven by sensors.

Three hypotheses are carried out to support our study: (H1) Q-Learning autonomous exploration is effective, achieving at least 75% coverage and outperform baseline-non learning agent. (H2) Reward shaping will improves robots performance compare to a semi-reward design. (H3) We would like to evaluate whether the trained policy can be generalise into different environments, if so, we would expect similar coverage across the environments.

II. INTRODUCTION - [COLIN CHONG, KENG XIANG TAN, HUMBLE UDOHNYAH]

Automation Exploration is one of the core fundamentals for mobile robots to explore unknown environments. Unlike classic navigation task, maps or prior knowledge of the environments are not available to the robots and it required robots to make accurate decision actions under uncertainty while gradually learning about the environment. This study is set on certain specific scenarios such as disaster response or planetary surface mission where human intervention are impossible due to nature of the environment is dangerous or not feasible for human to reach. Hence, in such settings, robots must rely on its sensor and decision-making process to move safely and efficiently.

To target this problem, all autonomous exploration need these 2 core modules - a decision making mechanism and a mapping method. We have selected **Reinforcement Learning**, specifically **Tabular-Q Learning** as our main core algorithm for exploration. The reason is such, reinforcement learning enables the robot to learn efficient navigation behaviours

directly from interactions, without the need of predefined rules or prior knowledge of the environment. As the robot explores, it will obtain rewards or penalties as a form of feedback, hence, allowing it to redefine its policy over time. On the mapping method, **Occupancy Grid Mapping (OGM)** has been selected, where it is able to provide probabilistic representation of free and occupied space so that exploration progress can be tracked and the robot can build a usable map while learning. By combining both of these modules, we aim to develop a navigation framework that is both adaptive to various environments while map-aware.

In this project, the goal is mainly focusing on investigating the effectiveness of Tabular Q-learning as the control mechanism for autonomous explorations in a fixed simulated world in webots. We will be comparing whether Q-Learning is able to outperform a non-learning baseline robot, how does the reward-penalty system is able to influence the the output policy and whether the policy is able to generalise across different environments. There won't be any implementations or new algorithms or improvement on certain algorithms, but it's rather a hands-on evaluation where we will try to pair different algorithms together and assess whether the pairing works and how reinforcement learning and mapping can be paired to support autonomous exploration in unknown environments.

III. METHODOLOGY

A. Reinforcement Learning (Tabular Q-Learning) - [Colin Chong]

Reinforcement Learning form the main control strategy in our study, due to its nature where it enables agents to learn through trial-and-error interaction with the environment, guided by reward signals rather than explicit supervision. **Tabular Q-Learning** is selected due to its transparent structure, where each state-action pair is clearly mapped out in the Q table. This allows a straightforward approach during the training and evaluation process, allowing clear identification of problems such as state-action mismatches, convergence anomalies, or abnormal behaviours.

1) **Environment, State Representation and Action States:** To enable tabular Q-learning, the robot environment is modelled using a fully discrete state space. Although the actual robot will be operating in a 3D environment, the learning process will be taken place in a 2D grid-world where it's a 1 to 1 clone of the Webots environment. The discrete training environment consists of a 12×12 grid, corresponding to a 3 m $\times$ 3 m floor area in Webots. Each cell in the grid corresponds to an unique space, and the robot will maintain

one of the 4 possible orientations: North, South, East or West. The orientations will determine the directions where the robot will make its forward movement. Formally, each state is defined as a tuple:

$$s = (x, y, \theta)$$

where:

- $x \in \{0, 1, 2, \dots, 11\}$ denote the row index,
- $y \in \{0, 1, 2, \dots, 11\}$ denote the column index,
- $\theta \in \{0, 1, 2, 3\}$ corresponds to the robot's orientation (North, East, South, West).

This give a total of:

$$n_{\text{states}} = 12 \times 12 \times 4 = 576 \text{ states}$$

To store the Q-values for each state efficiently, each state is mapped to a unique scalar-index in the Q-table. The encode function is such:

$$\text{state_index} = (x \times 12 + y) \times 4 + \theta$$

where $\mathbf{x}$ is the row, $\mathbf{y}$ is the column and θ is the orientation. This ensures that all possible combinations of position and orientation has its own row in the q-table. The inverse function (decoding) is such:

$$\begin{aligned} \text{base} &= \left\lfloor \frac{\text{state_index}}{4} \right\rfloor \\ \theta &= \text{state_index} \bmod 4 \\ x &= \left\lfloor \frac{\text{base}}{12} \right\rfloor \\ y &= \text{base} \bmod 12 \end{aligned}$$

where it recovers the tuple (x, y, θ) from a scalar index. This mapping allows each position–orientation pair can be directly linked to its corresponding Q-values and allow us to interpret the agent's current state in the grid.

The robot can choose from 3 discrete actions, where each of the action will update the states differently. The discrete action set is:

$$\text{Action} = \{0, 1, 2\}$$

where:

- **0: Move Forward** – The robot move 1 grid forward in the orientation it is facing,
- **1: Turn right** – The robot turn 90° clockwise without changing its position,
- **2: Turn left** – The robot turn 90° anti-clockwise without changing its position.

The actions updates the new position or the orientations according to the following logic:

$$\begin{aligned} (x', y') &= (x + \Delta x_\theta, y + \Delta y_\theta) && \text{(move forward)} \\ \theta' &= (\theta + 1) \bmod 4 && \text{(right turn)} \\ \theta' &= (\theta - 1) \bmod 4 && \text{(left turn)} \end{aligned}$$

where:

$$\theta = \text{original orientation,}$$

$$\Delta x_\theta = \text{change in the x-axis based on the orientation,}$$

$$\Delta y_\theta = \text{change in the y-axis based on the orientation.}$$

The directional unit vectors is defined as:

$$(\Delta x_\theta, \Delta y_\theta) = \begin{cases} (1, 0), & \theta = 0 \text{ (North)} \\ (0, 1), & \theta = 1 \text{ (East)} \\ (-1, 0), & \theta = 2 \text{ (South)} \\ (0, -1), & \theta = 3 \text{ (West)} \end{cases}$$

Note: Initially in the proposal there are 5 actions, however we had modify it to be 3 as backward movement and stopping are not feasible within Webots' realistic motion constraints.

2) Reward Function: We implement a reward system to encourage the robot for more explorations and prevents it from unsafe actions or repetitive explorations. The rewards implemented in the training environment as follow:

$$\text{Reward} = \begin{cases} +7.0, & \text{entering a new cell,} \\ -0.75, & \text{revisiting a cell visited before,} \\ -5.0, & \text{out of bounds collision,} \\ -3.0, & \text{obstacle collision,} \\ -0.1, & \text{turning left or right,} \\ -0.02, & \text{step cost for forward movement.} \end{cases}$$

We add penalty for a turning action to prevent the robot from spinning at the same cell and not exploring. Besides, to further promote exploration, we add a global visit counter for each cell. The global visit counter acts as a tracker to prevent that cell from over exploring and cause bias to unexplored cells. Each time a cell (x, y) is visited, an additional decay term is added:

$$r_{\text{decay}}(x, y) = \frac{30}{\text{visit_count}(x, y) + 1},$$

where at initial the bonus reward will be attractive (30 points) but eventually it will decrease as the cell is revisited more frequently. This dynamic shaping reduces the reward for repeatedly visiting the same region and provoke the agent to expand its coverage by visiting more unexplored cells.

3) Q-Learning Algorithm and Hyperparameters: The agent will learn an exploration policy using the standard tabular Q-Learning update rule. After executing an action and receiving a reward, the Q-value for the corresponding state–action pair is updated as follow:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

where $\mathbf{s}$ is the current state, $\mathbf{a}$ is the chosen action, $\mathbf{r}$ is the immediate reward, $\mathbf{s'}$ is the next state, α is the learning rate, γ is the discount factor, and $\max_{a'} Q(s', a')$ is the greedy estimate of the best future action. For the hyperparameters, the learning rate is set to $\alpha = 0.01$, and the discount factor is $\gamma = 0.98$. There values are chosen respectively to allow the robot learn slowly while prioritising long term benefits over short term gains while keeping it to be stable. During training, we use Epsilon-Greedy policy for actions selection where each action may either be the highest Q-value action or a random action with probability ϵ . The initial ϵ is 1 where it will slowly decay till $\epsilon = 0.1$ with a decay rate of 0.9998. While at evaluation, we will be using pure-greedy policy only,

where there will be no random actions. This means for each state, all the actions selected is the highest Q-value in the Q-table.

4) **Training:** The training is carried out offline, entirely in the discrete 12x12 grid world rather than in Webots. The reason this method is chosen is due to we would like to increase the training episodes, while not restrain by Webots physics constraint. Hence, when training offline, the physics logic will be ignore and the training state will be done at ideal state. We have let the robot to train for 100,000 episodes, while keeping the steps for each run to be 300. At the end of each episodes, the following metrics are recorded:

- Rewards obtain: total rewards per episode,
- training coverage: coverage under ϵ -greedy policy,
- evaluation coverage: coverage under greedy evaluation,
- visit_count: the number of times each cell is being visited

At the end of the training, the best performing Q-table is saved and visualised, together with other visuals plot such as reward vs episodes graph, training vs episodes coverage graph, eval vs episodes coverage graph, learning_progress graph and visit count heat map. These graph allows us to diagnosis any abnormal issues, evaluate the training performance and verify policy stability.

5) **Policy Deployment to Webots:** After training, we will select the best Q-table and import it into Webots controller. During executions state, the robot will operated in a continuous environment, however, its pose will be convert to discrete grid cells and orientation before querying the Q-table. The selected actions will then convert into motor commands: forward actions = both wheels moving at the same velocity, turning right or left = wheels are moving at opposite velocities.

IV. MAPPING(OGM AND A*) - [HUMBLE UDOHNYAH]

A. Simulation Platform

The experimental platform was Webots R2023b with the TurtleBot3 Burger robot equipped with the LDS-01 LiDAR sensor (360° coverage, 3.5 m range, 1° resolution). The robot's differential drive system has a wheel radius $r=0.033m$ and a wheelbase $L=0.160m$. A $2.7 \times 2.7m$ bounded arena with mixed obstacles serves as the test environment.

B. Occupancy Grid Mapping

1.) **Grid Representation** The environment is discretized into a 50×60 grid (0.05m resolution) where each cell stores occupancy probability $p \in [0, 1]$: $p = 0$ (free), $p = 0.5$ (unknown), $p = 1.0$ (occupied).

2.) **Coordinate Transformation** The World Coordinate Transformation (x_w, y_w) maps to grid indices via:

$$i = \lfloor x_w / \delta \rfloor + i_{origin}, \quad j = \lfloor y_w / \delta \rfloor + j_{origin} \quad (1)$$

where $\delta = 0.05m$ is the resolution.

3.) **Mapping Algorithm** For each LiDAR ray (r, θ):

$$\begin{aligned} \theta_{abs} &= \theta_{robot} + \theta_{sensor} \\ x_{end} &= x_{robot} + r \cdot \cos(\theta_{abs}) \\ y_{end} &= y_{robot} + r \cdot \sin(\theta_{abs}) \end{aligned} \quad (2)$$

Bresenham's line algorithm [1] identifies all cells between robot and endpoint with $O(n)$ complexity. Bayesian updates apply:

$$p(c) \leftarrow \begin{cases} \max(0, p(c) - \alpha_{free}) & \text{cells along ray} \\ \min(1, p(c) + \alpha_{occ}) & \text{endpoint cell} \end{cases} \quad (3)$$

where $\alpha_{free} = 0.01$ and $\alpha_{occ} = 0.05$. Asymmetric rates reflect higher confidence in obstacle detection.

Complexity: $O(n_{rays} \cdot d_{max})$ per scan where $n_{rays} = 360$, $d_{max} \approx 70$ cells, yielding approximately 25,200 operations per update.

C. Localisation Methods Odometry-based : Wheel encoder integration using differential drive kinematics:

$$\begin{aligned} \Delta s &= (\Delta s_L + \Delta s_R) / 2 \\ \Delta \theta &= (\Delta s_R - \Delta s_L) / L \\ x &\leftarrow x + \Delta s \cdot \cos(\theta + \Delta \theta / 2) \\ y &\leftarrow y + \Delta s \cdot \sin(\theta + \Delta \theta / 2) \end{aligned} \quad (4)$$

D. A* Path Planning After exploration, A* search [2] computes collision-free paths on the occupancy grid using Euclidean distance heuristic $h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$.

A* Path Planning Grid M , start s , goal g , threshold $\tau = 0.6$ Waypoint path P or $\emptyset$ $Q \leftarrow \{(h(s), s)\}$, $g(s) \leftarrow 0$, $CLOSED \leftarrow \emptyset$ $Q \neq \emptyset$ $n \leftarrow \text{extract_min}(Q)$ $n = g$ RECONSTRUCT_PATH(n) $CLOSED \leftarrow CLOSED \cup \{n\}$ neighbour n' of n $n' \in CLOSED$ or $M[n'] \geq \tau$ continue $cost \leftarrow g(n) + \text{dist}(n, n')$ $cost < g(n')$ $g(n') \leftarrow cost$ $f(n') \leftarrow cost + h(n')$ $Q \leftarrow Q \cup \{(f(n'), n')\}$ $\emptyset$

The algorithm uses 8-connectivity with diagonal cost $\sqrt{2}$ and orthogonal cost 1.0. Path simplification reduces waypoints by 80% by retaining every 5th point.

E. Visualisation and Data Export Real-time matplotlib visualization displays grayscale occupancy grid (black=occupied, white=free), robot position (red), trajectory (blue), and planned path (green). Update rates: mapping every 5 steps (320ms), display every 10 steps (640ms).

Metrics collected include coverage percentage, map entropy $H = -\sum [p \log_2 p + (1-p) \log_2 (1-p)]$, path length, and revisit counts. Maps export as NumPy .npy format.

V. EXPERIMENTS AND EVALUATION - [KENG XIANG TAN]

This section describes how the system is evaluated and how the three hypotheses H1–H3 are tested. The experiments have two parts. First, tabular Q-learning is trained and analysed in an idealised 12×12 gridworld; this setting is used to study the effect of different reward functions (H2). Second, the best Q-table learned on a grid that matches Webots world 1 is deployed on the simulated robot. We compare this Q-learning controller against a non-learning baseline on world 1 (H1) and then run the same pre-trained policy with the same LiDAR-based safety layer for 300 s in a more cluttered world 2 (H3).

A. Hypotheses

H1 asks whether learning improves exploration on the original map. On Webots world 1, a reactive LiDAR-based baseline

controller is compared against a controller that follows a tabular Q-learning policy trained offline on the corresponding gridworld. Both start from cell 0 and run for 300 s; H1 states that the Q-learning controller should achieve higher mean coverage.

H2 concerns reward design in the gridworld. Two Q-learning agents share the same state and action spaces, learning rate, discount factor and ϵ -greedy schedule, but use different rewards. The simple agent receives a first-visit bonus and collision penalties; revisits are almost neutral. The shaped agent keeps these terms but adds penalties for revisits and short loops to push the agent into unexplored areas. H2 claims that, with the same 100 000-episode budget, shaping yields higher best greedy coverage and a smoother evaluation curve.

H3 is about robustness to environmental change. The Q-table that performs best on world 1 is frozen and reused on world 2 without further training. The controller still uses the same LiDAR safety layer with threshold $D_{\text{SAFE}} = 0.25$ and is given 300 s to explore. H3 states that coverage will be lower and more variable in world 2 and that failures will often involve the robot becoming trapped near obstacles.

B. Experimental Setup

Both Webots environments are 12×12 arenas discretised into 144 grid cells indexed from 0 at the bottom left to 143 at the top right. Of these, 108 cells are traversable; the rest are walls or static obstacles. Continuous coordinates (x, y) are mapped linearly to integer row and column indices, and this mapping drives an internal visited-grid used for coverage. The obstacle layout is fixed across trials: world 1 has a long central wall and a few isolated platforms and rocks with wide corridors, whereas world 2 adds several rocks and narrow gaps, creating more local traps.

The robot is a differential-drive platform equipped with a 2D LiDAR, GPS, and compass. LiDAR scans are monitored in a narrow frontal sector and a simple time-to-collision estimate is computed from range and forward speed. If distance or time-to-collision falls below a threshold derived from D_{SAFE} , the controller switches from cruising to an avoidance mode that slows the robot and steers towards the more open side. Collisions are counted whenever the minimum LiDAR range enters and then leaves a short band close to the robot. GPS is projected onto the grid to update the visited array, and the compass heading is quantised into four orientations (north, east, south, west) that form part of the Q-learning state.

Two high-level controllers are evaluated. The baseline controller is purely reactive: at each control step it inspects the LiDAR scan, chooses between cruising and avoidance, and sets wheel speeds accordingly; it keeps no map and no long-term state. The Q-learning controller instead queries a tabular Q-table trained offline in a Python gridworld that mirrors world 1. States are triples (row, column, orientation) and actions are {forward, right, left}. In Webots, the controller reads the current grid state once per second, selects the greedy action from the Q-table, and converts it into a short motion primitive; the LiDAR safety layer runs in parallel and may override unsafe commands. All Webots trials start from cell 0 and run for at most 300 s.

C. Evaluation Metrics

In Webots, the main metric is *coverage*: the percentage of free cells visited at least once during a 300 s run. Whenever the projected grid cell is traversable, the corresponding entry in the visited array is set to one; coverage is the number of visited free cells divided by 108. As a safety indicator we also count *collisions*, incrementing a counter each time the minimum LiDAR distance enters the collision band after being outside it. Since all trials share the same time horizon, coverage and collision counts are directly comparable across controllers and worlds.

In the gridworld experiments, coverage is defined analogously. Each training or evaluation episode lasts for at most 300 steps and starts from state 0. Coverage is the fraction of distinct free cells visited within those 300 steps. For both reward designs we log coverage and total reward per training episode. For the shaped-reward agent we also run periodic greedy evaluation episodes, producing an evaluation-coverage curve and tracking the best greedy coverage over training.

D. Experimental Procedure

For H1, both controllers are run on world 1 for eight independent trials each. The robot always starts in cell 0 facing the positive y -axis. World geometry, safety parameters, and controller hyperparameters are fixed. During each 300 s run we log time, continuous position, grid state, coverage, cumulative collision count and command velocities at 10 Hz.

H2 is studied purely in the gridworld. Two Q-learning agents (simple and shaped reward) are trained on the same 12×12 map for 100 000 episodes with identical hyperparameters. Each episode starts in state 0 and ends after 300 steps or when the agent cannot move. After every episode we record coverage and total reward; for the shaped agent we periodically freeze learning and run a greedy evaluation episode.

For H3, the pre-trained Q-learning controller from H1 is deployed unchanged on both world 1 and world 2. We run eight 300 s trials on each world using the same logging setup. From these logs we compute mean and standard deviation of coverage and collisions and inspect trajectories for typical failure modes.

VI. RESULTS AND ANALYSIS - [KENG XIANG TAN, COLIN CHONG, HUMBLE UDOHNYAH]

A. Overall Performance Overview

Table I summarises the main Webots results. On world 1 the baseline covers on average only 27.1% of the 108 free cells in 300 s, whereas the Q-learning controller reaches 60.4%. When the same pre-trained policy is moved to world 2, mean coverage drops to 36.8%. Collision counts show the opposite trend: the baseline averages 8.6 contacts per run, while Q-learning yields 16.0 on world 1 and 14.5 on world 2.

Overall, learning brings a clear exploration gain on the original map at the cost of more frequent contacts. On the more cluttered map, the pre-trained policy still outperforms the baseline in coverage but loses a sizeable portion of its advantage and behaves less consistently.

TABLE I
EXPLORATION COVERAGE AND COLLISION COUNTS IN WEBOTS.

Condition	Mean coverage [%]	Mean collisions
Baseline, world 1	27.1	8.6
Q-learning, world 1	60.4	16.0
Q-learning, world 2	36.8	14.5

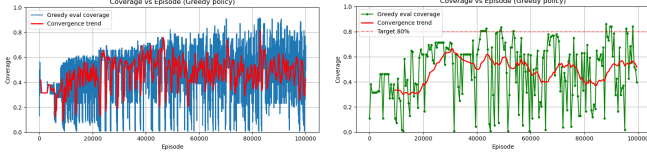


Fig. 1. Greedy evaluation coverage of the Q-learning agent in the 12×12 gridworld. Left: simple first-visit reward. Right: shaped reward.

B. Results for H1: Q-learning vs. baseline on world 1

H1 compares the Q-learning controller with the reactive baseline on world 1. The baseline is very stable: final coverage lies between 25.9% and 27.8%, with a mean of 27.1% (about 29 free cells) and tiny variance. It keeps the robot safe but tends to circle in a limited region once nearby cells are explored.

The Q-learning controller shows a different pattern. Across eight runs it usually ends near either 50.0% or 66.7% coverage, with a mean of 60.4% (about 65 cells). This is roughly a 33 percentage-point improvement over the baseline, corresponding to around 36 additional free cells visited within the same time budget. The price is a higher collision count: 16 events per run compared with 8–9 for the baseline, as the learned policy often skims close to obstacles and uses the safety layer as a soft boundary. These numbers support H1: on the training-like world, the tabular Q-learning policy offers much better exploration than the hand-crafted baseline while remaining within acceptable collision levels.

C. Results for H2: Effect of Reward Shaping

H2 looks at how reward shaping affects learning in the gridworld. Figure 1 plots the greedy evaluation coverage over training for the simple and shaped rewards.

With the simple reward, ϵ -greedy training coverage rises from roughly 0.35 to around 0.75–0.80, but the greedy evaluation curve is erratic: it swings between low coverage and occasional peaks near 0.9, and its moving average hovers around 0.45–0.55. Late in training the greedy policy often deteriorates again, suggesting that Q-values drift and the agent never settles on a stable exploration strategy.

The shaped-reward agent behaves more cleanly. Its evaluation curve climbs quickly and then stabilises. The best greedy coverage reaches 86.1% of free cells (about 95 out of 108) around episode 88000 and stays at or above 80% for long stretches. The moving average remains in the 0.70–0.80 range and catastrophic drops disappear; training coverage also varies less. Overall, reward shaping improves best-case performance, typical coverage, and stability, so H2 is strongly supported.

D. Results for H3: Robustness on a More Cluttered Map

H3 examines what happens when the pre-trained policy from world 1 is applied unchanged to world 2. On world 1 it reproduces the behaviour seen for H1: a mean coverage of 60.4%, standard deviation about 8.1 percentage points, around 65 visited cells, and 16 collisions per trial.

On world 2 the same controller struggles. Coverage ranges from 8.3% (only 9 cells) up to about 52.8%, with a mean of 36.8% and a larger standard deviation of 12.4 percentage points (about 40 cells on average). Collision counts remain in a similar range (roughly 14.5 per trial) but vary more between runs. The policy therefore loses roughly 40% of its coverage when moved to world 2, and its behaviour becomes less predictable. These results confirm H3: the Q-learning policy, trained on a single map, is not robust to the increased clutter and altered geometry of the second world.

E. Failure Cases and Qualitative Observations

Qualitative inspection of the trajectories helps explain the drop in performance on world 2.

A first failure mode occurs in local traps. Clusters of rocks form U- or L-shaped pockets; once inside, the robot repeatedly triggers the safety layer and oscillates between nearby obstacles without finding an escape, so coverage stops increasing after a small number of cells.

A second common failure involves narrow gaps and tight corridors. When approaching a narrow opening, the safety layer sometimes reacts symmetrically on both sides, causing the robot to waver in front of the gap. Because the Q-table was trained on wider corridors, there are few states in which “squeezing through” is strongly preferred, so the controller often backs off along the safer side rather than committing to the unseen area behind the gap.

These behaviours explain why coverage on world 2 is lower and more variable. The controller remains competent at local obstacle avoidance and rarely gets stuck in hard collisions, but the combination of local safety rules and a policy trained on a simpler geometry is not enough to reliably escape complex clutter. This feeds directly into the main limitation discussed in Section Limitations: the current approach relies on a fixed tabular policy without online adaptation or longer-term exploration planning.

VII. LIMITATIONS - [COLIN CHONG, KENG XIANG TAN, HUMBLE UDOHNYAH]

This section outlines the main limitations of our project, how the final outcome differs from our initial expectations, and the potential improvements that could be done to achieve better results.

1. Training offline vs training online: In this project, we have been training the q-learning agent in an ideal state, where each of actions is discrete and have a clear transition between each cell. However, the real Webots robot are actually operate in continuous space. This leads to the robot may not reach the intended cell in 1 action, causing misalignment to the grid and actions not being execute appropriately. Hence, the offline- training policy caused mismatch in terms of accuracy

in the final results. Suggestions: The q-learning agent should be trained in Webots, where we take account for the continuous movement and real world noise. Hence, the evaluation results will be more aligned to the training results.

2. Training instability and uneven learning across the environment: Although we have run the episodes at a large number, the agent does not explore all the cells completely yet, some of the cells (corners, region near obstacles) are less visited. As a result, the agent did not fully explore or learn all states, which contributes to the fluctuations observed in both the training and evaluation graphs despite an overall convergence trend. We suspected this problem arises due to the reward system may be either too weak, too harsh on punishment, or not differentiated evenly. The current reward between exploring new cells and revisiting cells might not be at optimum, hence the q-value updates become noisy and sensitive. Another possible reason is due to the less visited region are far less explored, hence it may cause incomplete training for these regions. When the robot gets into this region, the behaviour becomes inconsistent which causes the fluctuations mentioned earlier.

3. Policy cannot be generalised across environments: The Q-table is trained offline on a single idealised 12×12 grid map that corresponds to Webots world 1. The same Q-table is then reused unchanged in all Webots experiments. H3 shows that this setup does not generalise robustly to a more cluttered environment: when the same controller is deployed on world 2, mean coverage drops from 60.4% to 36.8%, and the variability between runs increases sharply (including one trial with only 8.3% coverage and 22 collisions). These results suggest that the learned policy is highly specialised to the obstacle configuration of world 1. The current work does not explore retraining on multiple maps, curriculum learning, or more expressive function approximators that might improve robustness across environments. An improvement could be done for evaluating this hypothesis by using Deep Q-learning (DQN) where it can generalise across states where it might help us prove the generalisation hypothesis.

4. Limited safety and collision behaviour: In world 1 the Q-learning controller achieves a much higher mean coverage than the hand-crafted baseline (60.4% vs. 27.1%), but this comes at the cost of a substantially higher collision count. On average the RL controller records 16 collisions per 300 s episode, roughly twice the 8–9 collisions of the baseline obstacle-avoidance controller. This indicates that the learned policy is more aggressive and tends to drive closer to walls and obstacles, relying on the LiDAR-based safety layer to override dangerous actions. The current evaluation only counts how many collisions occur; it does not measure how “near-misses”, oscillations under the safety layer, or physical damage would affect a real robot. As a result, the safety–exploration trade-off is only partially captured by the metrics.

5. Limited Metrics: Missing mapping accuracy $|M_{built} - M_{truth}|$, completeness (discovered/total area), and collision counts.

VIII. CONCLUSIONS - [KENG XIANG TAN, COLIN CHONG, HUMBLE UDOHNYAH]

This work establishes the simulation and mapping infrastructure for RL-based autonomous exploration research. The implemented occupancy grid mapper successfully constructs spatial representation from LiDAR using probabilistic Bayesian updates. A* path planning enables autonomous navigation using constructed maps.

Experiments on world 1 show that even this basic Q-learning setup clearly beats a hand-crafted reactive baseline. With the same safety layer and a 300 s time limit, the learned policy visits almost twice as many free cells, while collisions stay at a level that is acceptable in simulation. This supports H1: learning will improve exploration in the environment that was trained for and the safety layer is enough to keep the more aggressive behaviour under control.

The gridworld experiments highlight the role of reward design. A simple first-visit reward can reach reasonable coverage, but the greedy policy is unstable and sometimes becomes worse again late in training. Adding penalties for revisits and short loops makes learning much smoother, raises the best greedy coverage, and removes most of the sudden drops in performance. These results support H2 and show that a shaped reward is a key part of the exploration setup, not just a small tweak.

Applying the same Q-table to the more cluttered world 2 reveals its limits. Average coverage falls by about 40%, run-to-run variation increases, and the robot often gets stuck in U-shaped pockets or hesitates in front of narrow gaps. The policy is clearly tuned to the layout of world 1 and does not generalise well when the obstacle configuration changes, so H3 is confirmed: the current approach is not robust across different environments.

Overall, the results show both the strengths and weaknesses of tabular Q-learning for exploration. The method is easy to understand and implement, and it is useful for studying how different rewards shape behaviour. However, offline training in an ideal grid, a coarse state representation, and the lack of online adaptation limit performance when the world becomes more complex. The mapping and planning modules work, but in this project they are used mainly for visualisation, and mapping accuracy and long-term safety are not analysed in detail. Future work should address these gaps. Possible next steps include training directly in Webots with realistic dynamics and noise, training on several maps instead of one, and moving to function-approximation methods such as deep Q-learning to improve generalisation. It would also be valuable to measure mapping quality and safety more carefully, for example by comparing occupancy grids to ground-truth maps or by counting near misses and oscillations under the safety layer. These extensions would build on the current system and move it closer to use in real, partially unknown environments.

REFERENCES

- [1] J. E. Bresenham, “Algorithm for computer control of a digital plotter,” *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

- [3] A. Elfes, “Using occupancy grids for mobile robot perception and navigation,” *Computer*, vol. 22, no. 6, pp. 46–57, 1989.
- [4] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [5] Sutton, R. S., & Barto, A. G. (2014). Reinforcement Learning: An Introduction (Second edition, in progress). The MIT Press. <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2nded.pdf>
- [6] Watkins, C., H., & Dayan, P. (1992). Q-Learning. In Kluwer Academic Publishers, Machine Learning (Vol. 8, pp. 279–292) [Technical Note]. <https://www.gatsby.ucl.ac.uk/dayan/papers/cjch.pdf>
- [7] GeeksforGeeks. (2025, October 31). QLearning in reinforcement learning. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/>
- [8] Cyberbotics: doc. (n.d.). <https://cyberbotics.com/doc/guide/index>

Link to Repository: <https://git.cs.bham.ac.uk/ckc420/group-17-robotics-research-project>